

MSTCONNECT PH

SESSION 11 BOOTCAMPER HANDOUT

REST API Design, Postman Testing, and Capstone 2

Beginner Guided Lab Edition

Final output: A functional e-commerce REST API with MongoDB/Mongoose, product CRUD, search/filtering, validation and error handling, a saved Postman collection, README/API documentation, and a clean GitHub repository.

What You Will Build

In this handout you will take the database-backed API from Session 10 and organize it into a cleaner REST API. You will build predictable routes, separate routing from controller logic, validate product data, handle errors, test every endpoint in Postman, and document the finished API for another developer.

Student learning targets

- ☐ Explain what makes an API RESTful.
- ☐ Use GET, POST, PATCH, and DELETE appropriately.
- ☐ Distinguish route parameters, query parameters, and request bodies.
- ☐ Build Product CRUD controllers with Mongoose.
- ☐ Add search/category filtering through query parameters.
- ☐ Validate incoming product data and return useful status codes.
- ☐ Test success and failure cases in Postman.
- ☐ Create API documentation and a README that a classmate can follow.
- ☐ Produce a clean GitHub repository without secrets.

Capstone 2 minimum features

- ☐ Create, list, view, update, and delete products.
- ☐ Search or filter products by category or keyword.
- ☐ Use a category field on products or create/manage categories.
- ☐ Return consistent JSON responses and proper status codes.
- ☐ Validate inputs and handle errors cleanly.
- ☐ Provide a saved Postman collection and API documentation.

Scope reminder: Build the product API completely before adding optional users or orders. A complete, tested, documented core API is more valuable than several unfinished features.

REST API Basics for Beginners

An API is a controlled way for one program to request data or actions from another. In this project, Postman or a future React front end sends HTTP requests to Express. Express calls controller logic. Mongoose communicates with MongoDB. The server then returns JSON.

Request-response flow

```

Postman / React
  ↓ HTTP request
Express route
  ↓
Controller
  ↓
Mongoose model
  ↓
MongoDB
  ↑
JSON response
  
```

RESTful route design

```
GET /api/products
POST /api/products
GET /api/products/:id
PATCH /api/products/:id
DELETE /api/products/:id
```

Beginner rule: The URL says WHAT resource you are working with. The HTTP method says WHAT ACTION you want to perform.

Method	Meaning	Typical use	Expected success
GET	Read data	List/view products	200 OK
POST	Create data	Add a product	201 Created
PATCH	Partially update	Change price/stock	200 OK
DELETE	Remove data	Delete a product	200 OK

Common status codes

Code	Meaning	Example in this API
200	Success	GET/PATCH/DELETE worked
201	Created	POST saved a new product
400	Bad request	Missing/invalid input
404	Not found	Product does not exist
500	Server error	Unexpected backend failure

How Data Enters an API Request

Route parameter - identify one resource

```
GET /api/products/64abc123

router.get('/:id', getProductById);

// inside controller
const id = req.params.id;
```

Use a route parameter when the value identifies one exact resource. Here, `:id` identifies one product.

Query parameter - filter/search a collection

```
GET /api/products?category=Laptop&search=gaming

const { category, search } = req.query;
```

Use query parameters for optional filtering, searching, sorting, or pagination. You are still requesting the products collection; the query changes which products are returned.

Request body - send new or changed data

```
POST /api/products
Content-Type: application/json
```

```
{
  "name": "Mechanical Keyboard",
  "description": "RGB keyboard",
  "price": 1850,
  "category": "Accessories",
  "stock": 12
}
```

Important: `app.use(express.json())` must run before routes that need `req.body`. In Postman, choose Body → raw → JSON.

Guided Lab Part A - Project Setup

Recommended project structure

```
capstone-2-ecommerce-api/
├── src/
│   ├── config/
│   │   └── db.js
│   ├── controllers/
│   │   └── productController.js
│   ├── middleware/
│   │   └── errorHandler.js
│   ├── models/
│   │   └── Product.js
│   ├── routes/
│   │   └── productRoutes.js
│   └── server.js
├── .env
├── .gitignore
├── package.json
└── README.md
```

Create the project

PowerShell or Terminal

```
mkdir capstone-2-ecommerce-api
cd capstone-2-ecommerce-api
npm init -y
npm install express mongoose dotenv cors
npm install --save-dev nodemon
```

Add npm scripts

package.json

```
"scripts": {
  "dev": "nodemon src/server.js",
  "start": "node src/server.js"
}
```

Environment files

```
# .env
PORT=5000
MONGO_URI=your_real_connection_string
```

```
# .gitignore
node_modules/
.env
```

```
# .env.example
PORT=5000
MONGO_URI=your_mongodb_connection_string_here
```

Security check: Never put the real MONGO_URI in README or GitHub. Before your first push, confirm .env does not appear in staged files.

Checkpoint A

- ☐ package.json exists
- ☐ Dependencies installed
- ☐ Folder structure created
- ☐ .env created
- ☐ .env ignored by Git
- ☐ .env.example uses placeholders only

Guided Lab Part B - Database Connection and Product Model

src/config/db.js

```
const mongoose = require('mongoose');

async function connectDB() {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log('MongoDB connected');
  } catch (error) {
    console.error('MongoDB connection failed:', error.message);
    process.exit(1);
  }
}

module.exports = connectDB;
```

If the database cannot connect, stop and fix the connection first. CRUD routes cannot work reliably without MongoDB.

src/models/Product.js

```
const mongoose = require('mongoose');

const productSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
```

```

description: { type: String, required: true },
price: { type: Number, required: true, min: 0 },
category: { type: String, required: true, trim: true },
stock: { type: Number, required: true, min: 0 },
imageUrl: { type: String },
isActive: { type: Boolean, default: true }
}, { timestamps: true });

```

```
module.exports = mongoose.model('Product', productSchema);
```

Understand the schema

Syntax	Meaning	Why it matters
required: true	Field must be present	Prevents incomplete products
trim: true	Remove outside spaces	Keeps names/categories clean
min: 0	No negative numbers	Protects price/stock
default: true	Use true if omitted	New product starts active
timestamps: true	Add createdAt/updatedAt	Tracks record history

Checkpoint B

- ☐ MongoDB connected message appears
- ☐ Product schema file loads without syntax error
- ☐ Price rejects negative values
- ☐ Stock rejects negative values

Guided Lab Part C - Product Routes

src/routes/productRoutes.js

```

const express = require('express');
const router = express.Router();

const {
  createProduct,
  getProducts,
  getProductById,
  updateProduct,
  deleteProduct
} = require('../controllers/productController');

router.route('/')
  .get(getProducts)
  .post(createProduct);

router.route('/:id')
  .get(getProductById)
  .patch(updateProduct)
  .delete(deleteProduct);

module.exports = router;

```

Why this file stays simple: The router should show the API map. Database logic belongs in controllers so routes remain readable and easy to document.

Guided Lab Part D - Create and List Products

Create Product - POST /api/products

```
const Product = require('../models/Product');

exports.createProduct = async (req, res, next) => {
  try {
    const { name, price, category, stock } = req.body;

    if (!name || price === undefined || !category || stock === undefined) {
      return res.status(400).json({
        success: false,
        message: 'Name, price, category, and stock are required'
      });
    }

    if (Number(price) < 0 || Number(stock) < 0) {
      return res.status(400).json({
        success: false,
        message: 'Price and stock cannot be negative'
      });
    }

    const product = await Product.create(req.body);

    res.status(201).json({
      success: true,
      message: 'Product created',
      data: product
    });
  } catch (error) {
    next(error);
  }
};
```

Why price === undefined?: Using !price would treat 0 as missing because JavaScript considers 0 falsy. Checking === undefined distinguishes “not sent” from the number zero.

List Products - GET /api/products

```
exports.getProducts = async (req, res, next) => {
  try {
    const { category, search } = req.query;
    const filter = {};

    if (category) {
      filter.category = category;
    }

    if (search) {
      filter.name = { $regex: search, $options: 'i' };
    }

    const products = await Product.find(filter);

    res.status(200).json({
      success: true,
      count: products.length,
      data: products
    });
  } catch (error) {
    next(error);
  }
};
```

Try these URLs

```
GET /api/products
GET /api/products?category=Accessories
GET /api/products?search=mouse
GET /api/products?category=Accessories&search=wireless
```

Guided Lab Part E - View, Update, and Delete

View one product

```
exports.getProductById = async (req, res, next) => {
  try {
    const product = await Product.findById(req.params.id);

    if (!product) {
      return res.status(404).json({
        success: false,
        message: 'Product not found'
      });
    }

    res.status(200).json({ success: true, data: product });
  } catch (error) {
    next(error);
  }
};
```

Update one product

```
exports.updateProduct = async (req, res, next) => {
  try {
    const product = await Product.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true }
    );

    if (!product) {
      return res.status(404).json({
        success: false,
        message: 'Product not found'
      });
    }

    res.status(200).json({
      success: true,
      message: 'Product updated',
      data: product
    });
  } catch (error) {
    next(error);
  }
};
```

Important update options: `new: true` returns the updated version. `runValidators: true` applies schema validation during the update.

Delete one product

```
exports.deleteProduct = async (req, res, next) => {
  try {
    const product = await Product.findByIdAndDelete(req.params.id);

    if (!product) {
      return res.status(404).json({
        success: false,
        message: 'Product not found'
      });
    }

    res.status(200).json({
      success: true,
      message: 'Product deleted'
    });
  } catch (error) {
    next(error);
  }
};
```

Guided Lab Part F - Error Handling and server.js

Central error middleware

src/middleware/errorHandler.js

```
function errorHandler(err, req, res, next) {
  console.error(err);

  let statusCode = res.statusCode === 200 ? 500 : res.statusCode;
  let message = err.message || 'Server error';

  if (err.name === 'CastError') {
    statusCode = 400;
    message = 'Invalid ID format';
  }

  res.status(statusCode).json({
    success: false,
    message
  });
}

module.exports = errorHandler;
```

Wire everything together

src/server.js

```
require('dotenv').config();

const express = require('express');
const cors = require('cors');
const connectDB = require('./config/db');
const productRoutes = require('./routes/productRoutes');
const errorHandler = require('./middleware/errorHandler');

connectDB();

const app = express();

app.use(cors());
app.use(express.json());
```

```
app.get('/', (req, res) => {
  res.json({ message: 'E-commerce API is running' });
});

app.use('/api/products', productRoutes);

app.use(errorHandler);

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Middleware order matters: `express.json()` must come before routes that read `req.body`. `errorHandler` should come after routes that may call `next(error)`.

Checkpoint C - API starts

- ☐ npm run dev starts server
- ☐ MongoDB connected
- ☐ GET / returns API message
- ☐ No blocking terminal errors

10. Guided Lab Part G - Build the Postman Collection

1. Create a new Postman collection named MSTCONNECT Capstone 2 API.
2. Create a folder named Products.
3. Add a collection/environment variable named `baseUrl` with value `http://localhost:5000`.
4. Save Create Product, List Products, View Product by ID, Update Product, and Delete Product.
5. Save error-case requests instead of deleting them; they prove validation works.

Use a base URL variable

```
{{baseUrl}}/api/products
```

Recommended request names

- ☐ Create Product
- ☐ List Products
- ☐ View Product by ID
- ☐ Update Product
- ☐ Delete Product
- ☐ Create Product - Missing Name
- ☐ Update Product - Negative Price
- ☐ View Product - Invalid ID
- ☐ View Product - Missing Product

Guided Lab Part H - Test CRUD Step by Step

A. CREATE

```
POST {{baseUrl}}/api/products
Body → raw → JSON
```

```
{
  "name": "Mechanical Keyboard",
  "description": "RGB mechanical keyboard",
  "price": 1850,
  "category": "Accessories",
  "stock": 12
}
```

Expected: 201 Created

Copy the returned `_id`. You will use it for Read One, Update, and Delete.

B. READ ALL

```
GET {{baseUrl}}/api/products
Expected: 200 OK
Expected body: success, count, and data array
```

C. READ ONE

```
GET {{baseUrl}}/api/products/<product-id>
Expected: 200 OK
```

D. UPDATE

```
PATCH {{baseUrl}}/api/products/<product-id>
Body:
{
  "price": 1700,
  "stock": 8
}
Expected: 200 OK
```

GET the product again. Verify price and stock changed while name, description, and category stayed the same.

E. DELETE

```
DELETE {{baseUrl}}/api/products/<product-id>
Expected: 200 OK

Then:
GET {{baseUrl}}/api/products/<same-product-id>
Expected: 404 Not Found
```

Strong CRUD proof: POST → copy ID → GET → PATCH → GET again → DELETE → GET again and receive 404.

Guided Lab Part I - Test Failure Cases

Test	What to send	Expected
Missing required field	POST without name	400
Negative price	POST/PATCH price -500	400
Malformed ID	GET /api/products/abc	Handled invalid ID, e.g. 400
Missing product	GET valid but nonexistent ID	404
Delete missing product	DELETE nonexistent ID	404

Why failure tests matter: A professional API must behave predictably when requests are wrong, not only when they are perfect.

Search and Filter Practice

The same GET /api/products route can support different list requests by reading req.query. Do not create a separate route for every possible filter.

```
GET {{baseUrl}}/api/products?category=Accessories
GET {{baseUrl}}/api/products?search=keyboard
GET {{baseUrl}}/api/products?category=Accessories&search=wireless
```

Practice task

- ☐ Create at least 5 products
- ☐ Use at least 2 categories
- ☐ Verify category filter returns only matching category
- ☐ Verify search ignores capitalization
- ☐ Verify no query parameters still return all products

Guided Lab Part J - Write the API Documentation

Your README should allow a classmate to clone, install, configure, run, and test the API without asking you for missing steps.

README starter template

```
# Capstone 2 - E-Commerce REST API

## Description
A REST API for managing e-commerce products using Node.js, Express, MongoDB, and Mongoose.

## Technologies
- Node.js
- Express.js
- MongoDB
- Mongoose
- Postman

## Installation
1. Clone the repository
2. Run: npm install
3. Create .env:
  PORT=5000
  MONGO_URI=your_connection_string
4. Run: npm run dev

## Base URL
http://localhost:5000

## Product Endpoints
- POST /api/products
- GET /api/products
- GET /api/products/:id
- PATCH /api/products/:id
- DELETE /api/products/:id

## Search / Filter
GET /api/products?category=Accessories
GET /api/products?search=mouse

## Testing
Import/use the Postman collection: MSTCONNECT Capstone 2 API
```

Document each endpoint

```
### Create Product
POST /api/products

Purpose: Create a product.

Body:
{
  "name": "Mechanical Keyboard",
  "description": "RGB keyboard",
  "price": 1850,
  "category": "Accessories",
  "stock": 12
}

Success: 201 Created
Possible errors: 400 Bad Request
```

Never do this: Do not place the real MONGO_URI in README. Show the variable name and placeholder only.

Git and GitHub Submission Workflow

```
git status
git add .
git status
git commit -m "Build product REST API with validation"
git push
```

Before pushing

- ☐ .env is not staged
- ☐ node_modules is ignored
- ☐ README is updated
- ☐ API starts successfully
- ☐ Postman tests are saved
- ☐ No passwords/tokens appear in source files

Beginner Troubleshooting Guide

Problem	Likely reason	Check first
req.body is undefined	JSON middleware/body type issue	app.use(express.json()) and Postman raw JSON
Cannot GET /api/products	Route mount/path mismatch	server.js mount + router path
MongoDB connection failed	URI/credentials/network	MONGO_URI and Atlas settings
CastError	Malformed ObjectId	ID used in URL
Validation not working on PATCH	Missing update validation	runValidators: true
Port already in use	Another server process	Stop old server/change port
CORS browser error	Cross-origin browser restriction	cors middleware/config

Debugging routine

1. Read the terminal error.
2. Confirm server and MongoDB are running.
3. Verify request method and URL.
4. Verify headers/body.
5. Log req.params, req.query, or req.body temporarily.
6. Test one endpoint at a time.
7. Change one thing, then retest the same request.

Capstone 2 Final Checklist

Functional API

- ☐ POST creates product
- ☐ GET lists products
- ☐ GET by ID returns one product
- ☐ PATCH updates product
- ☐ DELETE removes product
- ☐ Search/filter works
- ☐ Validation rejects bad data
- ☐ 404/invalid ID handling works

Postman

- ☐ Collection exists
- ☐ baseUrl variable used
- ☐ CRUD requests saved
- ☐ Failure cases saved
- ☐ Example bodies included
- ☐ Requests have clear names

Documentation

- ☐ README description
- ☐ Technology list
- ☐ Installation steps
- ☐ .env variable names only
- ☐ Endpoint list
- ☐ Sample request body
- ☐ Testing instructions

GitHub

- ☐ Public/required repository created
- ☐ Clean commits
- ☐ No .env
- ☐ No node_modules
- ☐ Latest code pushed

Quick Reference

Need	Code / method
Create document	<code>Product.create(req.body)</code>
List documents	<code>Product.find(filter)</code>
Find one	<code>Product.findById(req.params.id)</code>
Update one	<code>Product.findByIdAndUpdate(id, body, {new:true, runValidators:true})</code>
Delete one	<code>Product.findByIdAndDelete(id)</code>
Route value	<code>req.params.id</code>
Filter/search values	<code>req.query</code>
JSON body	<code>req.body</code>
Send JSON	<code>res.status(code).json({...})</code>
Pass error	<code>next(error)</code>

Final goal: Your output is not just “code that runs.” It is a functional, tested, documented e-commerce API that another developer can understand and use.